Restaurant POS Project Workflow

Frontend cart, table, menu, kitchen, order, and admin report flows with backend/API and MySQL data lifecycle.

Area	Implementation
Frontend	React POS UI: tables, menu, cart, kitchen, orders, report
Backend	Spring Boot REST API with JWT security and service layer
Database	MySQL schema: users, tables, orders, cart items, notes, menu data
Generated	May 09, 2026

1. System Overview

The application is a role-based restaurant POS. The React frontend calls Spring Boot REST endpoints through API helper modules. The backend stores the authoritative state in MySQL and uses JWT-based authentication to control access.

Layer	Responsibilities	Important Files
React frontend	Role tabs, table layout, menu cards, cart, kitchen tickets, order modal, report view.	pos-front/src/features/pos/pages/POSPage.jsx; pos-front/src/features/pos/components/*
API client	Adds auth token, sends CRUD requests, maps frontend actions to backend endpoints.	pos-front/src/api/*.js; pos-front/src/features/auth/api/authApi.js
Spring controllers	Expose REST endpoints for auth, tables, menu, cart, orders, users, customers.	pos-back/restaurant-pos/src/main/java/.../controller/*
Service layer	Business rules: customer table restriction, table freeing, totals, checkout, close code.	OrderService.java, CartItemService.java, UserService.java
MySQL	Stores users, table position/status, orders, menu items, cart items, item notes.	restaurant_pos database

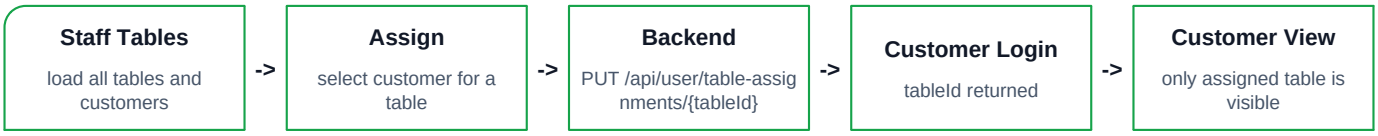
Role Access Summary

Role	Visible Sections	Main Actions
Admin	Tables, Menu, Cart, Kitchen, Orders, Report	Full POS operation plus report access, menu/table/order management.
Waiter/Cashier	Tables, Menu, Cart, Kitchen, Orders	Open tables, place orders, send kitchen items, checkout, edit orders.
Kitchen	Kitchen only	See sent items, mark finished, revert finished item when needed.
Customer	Tables, Menu, Cart	Open assigned table only, order menu items, view pending/sent cart state.

2. Authentication And Role Routing

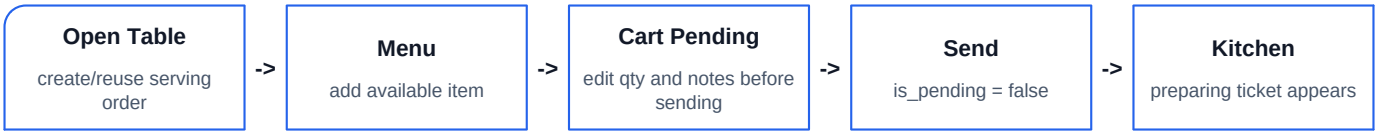
- Admin gets the report tab; kitchen gets only the kitchen tab.
- Customer starts in the table flow and only loads the assigned table.
- If the customer tries to open a different table, the backend rejects it.
- The frontend stores only the session token and display role data; backend keeps the source of truth.

3. Table And Customer Assignment Workflow



- Staff can drag table cards by long-holding the whole card, not a hidden drag button.
- Table position saves through the table update API and is stored as posX/posY.
- Customer assignments are stored on users.table_id.
- Opening a table creates or reuses a serving order and marks that table occupied.

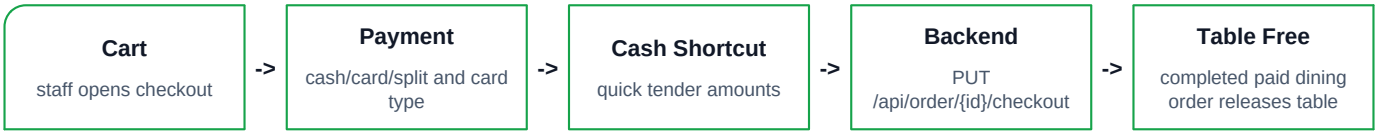
4. Ordering, Cart, And Kitchen Workflow



Step	What Happens	Data Updated
Open table	OrderService returns active serving order or creates a new one.	orders, res_tables.table_status
Add item	Cart item uses menu item snapshots so later menu price changes do not change the order.	cart_items.name_snapshot, price_snapshot
Add note/addition	Staff can add many notes; each note may have a price. Customers cannot add notes.	cart_item_notes
Send item	Item moves from pending cart to sent/preparing state.	cart_items.is_pending, sent_at
Kitchen finish	Kitchen marks item finished and can revert it if pressed by mistake.	cart_items.is_finished, finished_at

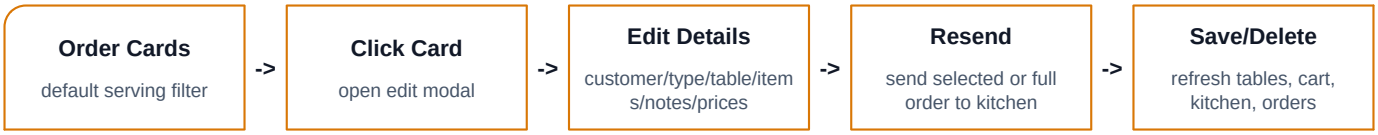
Kitchen finished status does not free a table. A table is freed by checkout, order delete, close order, or changing an active dining order away from dining.

5. Checkout Workflow



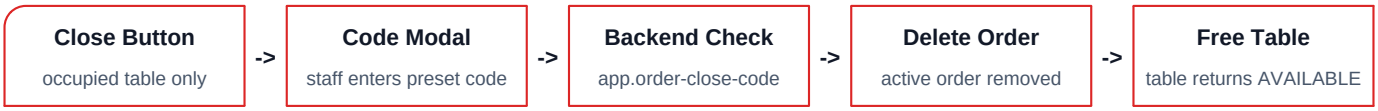
- Customer role has no checkout button; customers can only place and review their order.
- Decimal money inputs are constrained to two decimal places in the UI.
- Checkout sets order_status COMPLETED and payment_status PAID.

6. Orders Section Workflow



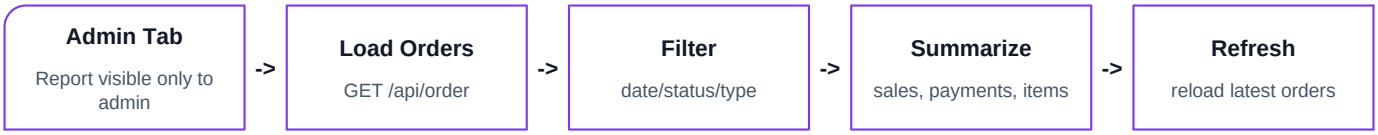
Feature	Workflow
Search in modal	Items can be searched by name, status, price, or note to handle long orders.
Select visible	Visible items can be selected and resent to the kitchen if a ticket was missed.
Edit order type	Dining can become To Go or Delivery; the previous table is freed.
Delete item	Item delete recalculates totals and refreshes the modal/order list.
Delete order	Two-step confirmation protects against accidental deletion; table is freed if applicable.

7. Close Order And Delete Safety



- The close code is stored in backend application.properties, not frontend code.
- Delete order from the orders modal also frees the table for dining orders.
- Close order is for force-closing an active table order; checkout is the normal paid flow.

8. Admin Report Workflow



- Report calculations are frontend summaries from the orders endpoint.
- Metrics include gross sales, paid sales, open balance, average order, counts, type mix, payment mix, and top items.
- Kitchen, customer, waiter, and cashier roles do not see the report tab.

9. Backend Endpoint Map

Domain	Important Endpoints
Auth	POST /api/auth/login
Tables	GET/POST/PUT/DELETE /api/table; GET /api/table/{id}
Users	GET /api/user/customers; PUT /api/user/table-assignments/{tableId}
Menu	GET/POST/PUT/DELETE /api/menuItem; modifier CRUD under /api/menuItemModifier
Cart	POST /api/cart; PUT /api/cart/{id}; send/finish/revert; notes CRUD
Orders	GET /api/order; open/close table; update; customer update; move-table; checkout; delete
Customers	GET/POST/PUT/DELETE /api/customer

10. Data Lifecycle Rules

Rule	Why It Matters
Orders snapshot item names and prices.	Changing the menu later does not rewrite historical or active order prices.
Notes are separate rows with optional prices.	One cart item can have many human-written notes/additions.
Dining order owns table occupancy.	Checkout, delete, close, or changing to non-dining frees the table.
Kitchen finish is item-level only.	Food completion does not imply payment or table release.
Customer table assignment is user-level.	Customer accounts can only open the table assigned by staff/admin.

11. Current Demo Test Data

The local database was reset for testing with the following seeded accounts. All seeded account passwords are 1234.

Role	Accounts
Admin	admin1, admin2, admin3
Waiter	waiter1, waiter2
Kitchen	kitchen1
Customer	customer1, customer2, customer3, customer4, customer5, customer6

Customer	Assigned Table
customer1	T1
customer2	T2
customer3	T3
customer4	T4
customer5	T5
customer6	T6

Seeded workflow data includes serving dining orders on T1 and T2, completed orders for report testing, one cancelled order, menu categories, modifiers, and kitchen-visible sent items.

Quick Manual Regression Path

- Login admin1 -> confirm Tables, Menu, Cart, Kitchen, Orders, Report.
- Long-hold a table card -> drag -> position persists after refresh.
- Assign customer3 to a table -> login customer3 -> only that table appears.
- Open a table -> add menu item -> send to kitchen -> kitchen1 sees ticket.
- Mark finished -> revert finished -> item returns to preparing.
- Checkout a dining order -> table becomes AVAILABLE.
- Delete or close a table order -> table becomes AVAILABLE.